

The Agent's Signature: Identity &
Law in the Age of AI
Comprehensive Illustrated Edition

Mauricio A. Fernandez Fernandez

December 2025

Contents

1	The Agent’s Signature	1
1.1	Identity, Authorization & Law in the Age of Autonomous AI	1
2	Preface: Why This Book Matters Now	3
3	Table of Contents	5
4	Part I: Foundations	9
5	Chapter 1: The OAuth Trap — Why Access Tokens Aren’t Enough	11
5.1	1.1 The Fundamental Distinction: Access vs. Authority	12
5.2	1.2 Real-World Failure Modes	15
5.2.1	Case Study 1: The Autonomous Procurement Agent	15
5.2.2	Case Study 2: The Healthcare AI Advocate	17
5.3	1.3 The Solution: Proof of Authorization (PoA) .	18
6	Chapter 2: The Architecture of Trust	21
6.1	2.1 The Three Pillars	21
6.2	2.2 Pillar 1: Identity (AAP-001)	22
6.2.1	The Entity Profile	22

6.2.2	The Authorization Chain	25
6.3	2.3 Pillar 2: Delegation (AAP-002)	27
6.3.1	The Proof of Authorization Token	27
6.3.2	Real-World Example: German Corporate Procurement	29
6.4	2.4 Pillar 3: Resilience (Degraded Mode)	37
7	Chapter 3: The Technical Implementation	41
7.1	3.1 System Architecture	41
7.2	3.2 Core Data Flows	42
7.2.1	Flow 1: PoA Token Issuance	42
7.2.2	Flow 2: Authorization Decision	43
7.3	3.3 Cryptographic Architecture	45
7.3.1	Supported Algorithms	45
7.3.2	Multi-Signature Support	46
7.4	3.4 Revocation Transparency	47
8	Chapter 4: Legal Frameworks Across Jurisdic- tions	51
8.1	4.1 The Universal Challenge	52
8.2	4.2 Germany: The Commercial Register Model . .	53
8.2.1	German Implementation Pattern	54
8.2.2	Key Legal Provisions	54
8.3	4.3 United States: The Apparent Authority Model	55
8.3.1	Implications for AgentAuth	56
8.4	4.4 European Union: The eIDAS Framework . . .	57
8.4.1	eIDAS Trust Levels	57
8.4.2	AgentAuth eIDAS Integration	58
9	Chapter 5: Building Trustworthy Agents	61
9.1	5.1 The Agent Capability Levels	61
9.1.1	Capability Level Requirements	62
9.2	5.2 Implementing a Procurement Agent	64
9.2.1	Step 1: Define the Entity Profile	64
9.2.2	Step 2: Issue PoA Token	66

9.2.3	Step 3: Execute Authorized Transaction . . .	69
9.3	5.3 Verification Workflow	71
10	Chapter 6: Operational Excellence	75
10.1	6.1 Observability	75
10.1.1	Key Metrics	76
10.2	6.2 Audit & Compliance	78
10.2.1	Cryptographic Audit Trail	78
10.2.2	External Anchoring	80
11	Chapter 7: Advanced Patterns	83
11.1	7.1 Multi-Party Authorization	83
11.2	7.2 Conditional Delegation	84
11.3	7.3 Cascade Revocation	86
12	Appendix A: Quick Reference	89
12.1	A.1 PoA Token Claims	89
12.2	A.2 Error Codes	90
12.3	A.3 API Endpoints	91
13	Appendix B: Glossary	93
14	About the AgentAuth Project	97
14.1	Contributors	98
15	Chapter 8: Real-World Case Studies	101
15.1	8.1 Case Study: German Manufacturing GmbH . . .	102
15.1.1	Background	102
15.1.2	The Challenge	103
15.1.3	The AgentAuth Implementation	104
15.1.4	Results (6 Months)	106
15.1.5	Lessons Learned	107
15.2	8.2 Case Study: US Fintech Startup	108
15.2.1	Background	108
15.2.2	The Challenge	108
15.2.3	The AgentAuth Implementation	109

15.2.4	Results (3 Months)	111
15.2.5	Key Insight	111
15.3	8.3 Case Study: Healthcare AI Advocate	112
15.3.1	Background	112
15.3.2	The Legal Challenge	112
15.3.3	The AgentAuth Implementation	113
15.3.4	Results (12 Months)	115
16	Chapter 9: Integration Patterns	117
16.1	9.1 Integrating with Existing OAuth Systems	118
16.1.1	Pattern 1: OAuth + AgentAuth Hybrid	119
16.1.2	Implementation	119
16.1.3	Pattern 2: OAuth Token Upgrade	122
16.2	9.2 Integration with Cloud Providers	125
16.2.1	AWS Integration	125
16.2.2	Azure Integration	127
16.3	9.3 Database Integration	129
16.3.1	PostgreSQL Row-Level Security	129
17	Chapter 10: Troubleshooting & FAQ	133
17.1	10.1 Common Issues	134
17.1.1	Issue: “PoA Verification Failed - Chain Invalid”	134
17.1.2	Issue: “Transaction Exceeds Liability Cap”	136
17.1.3	Issue: “Revocation Check Timeout”	138
17.2	10.2 Frequently Asked Questions	141
17.2.1	Q: Can AgentAuth work without a central server?	141
17.2.2	Q: How does AgentAuth handle agent key rotation?	142
17.2.3	Q: What happens if the principal’s key is compromised?	144
17.2.4	Q: Can PoA tokens be transferred?	144
18	Chapter 11: Future Directions	147

18.1	11.1 Post-Quantum Cryptography	148
	18.1.1 Current Status	149
	18.1.2 Roadmap	149
	18.1.3 Implementation Guide	151
18.2	11.2 Zero-Knowledge Proofs	152
18.3	11.3 Multi-Agent Coordination	154
19	Conclusion: The Path Forward	157

Chapter 1

The Agent's Signature

1.1 Identity, Authorization & Law in the Age of Autonomous AI

A Comprehensive Technical & Legal Framework

By Mauricio A. Fernandez Fernandez

*“In the age of autonomous agents, every transaction
is a legal act, every algorithm a potential fiduciary,
and every cryptographic signature carries the weight
of centuries of contract law.”*

Chapter 2

Preface: Why This Book

Matters Now

The year is 2025. Every major technology company has deployed thousands of AI agents handling customer service, code generation, supply chain optimization, and financial transactions. Yet

a fundamental question remains unanswered:

When an AI agent signs a contract, who is liable?

This isn't a hypothetical. Bloomberg estimates that over \$4 trillion in transactions are now initiated by autonomous systems annually. McKinsey projects this will reach \$25 trillion by 2030. Yet our authorization infrastructure—OAuth 2.0, originally designed for “Login with Google” buttons—was never built for agents that act, not just access.

This book provides the definitive technical and legal framework for building **trustworthy autonomous agents**. It is based on the **AgentAuth** open-source implementation (v1.0.0, December 2025), which implements the AAP-001 and AAP-002 protocols.

Chapter 3

Table of Contents

- Preface: Why This Book Matters Now
- Part I: Foundations
- Chapter 1: The OAuth Trap — Why Access Tokens Aren't
Enough
 - 1.1 The Fundamental Distinction: Access vs. Author-

ity

- 1.2 Real-World Failure Modes
- 1.3 The Solution: Proof of Authorization (PoA)
- Chapter 2: The Architecture of Trust
 - 2.1 The Three Pillars
 - 2.2 Pillar 1: Identity (AAP-001)
 - 2.3 Pillar 2: Delegation (AAP-002)
 - 2.4 Pillar 3: Resilience (Degraded Mode)
- Chapter 3: The Technical Implementation
 - 3.1 System Architecture
 - 3.2 Core Data Flows
 - 3.3 Cryptographic Architecture
 - 3.4 Revocation Transparency
- Chapter 4: Legal Frameworks Across Jurisdictions

- 4.1 The Universal Challenge
- 4.2 Germany: The Commercial Register Model
- 4.3 United States: The Apparent Authority Model
- 4.4 European Union: The eIDAS Framework
- Chapter 5: Building Trustworthy Agents
 - 5.1 The Agent Capability Levels
 - 5.2 Implementing a Procurement Agent
 - 5.3 Verification Workflow
- Chapter 6: Operational Excellence
 - 6.1 Observability
 - 6.2 Audit & Compliance
- Chapter 7: Advanced Patterns
 - 7.1 Multi-Party Authorization
 - 7.2 Conditional Delegation

- 7.3 Cascade Revocation
 - Appendix A: Quick Reference
 - A.1 PoA Token Claims
 - A.2 Error Codes
 - A.3 API Endpoints
 - Appendix B: Glossary
 - About the AgentAuth Project
 - Contributors
-

Chapter 4

Part I: Foundations

Chapter 5

Chapter 1: The OAuth

Trap — Why Access

Tokens Aren't Enough

5.1 1.1 The Fundamental Distinction:

Access vs. Authority

Every digital system today operates on a simple premise: **au-**
thenticate the user, then **authorize** their actions. OAuth 2.0

5.1. 1.1 THE FUNDAMENTAL DISTINCTION: ACCESS VS. AUTHORITY¹³

perfected this model for web applications. But it has a fatal flaw.

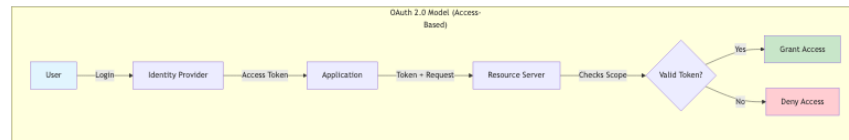


Figure 5.1: Diagram 1

Figure 1: Architecture diagram

Figure 1.1: The OAuth 2.0 access model — simple, effective, but legally blind

This model answers: *“Can this entity access this resource?”*

It does **not** answer: - *“Is this entity legally empowered to bind*

its principal?” - *“Does this action fall within fiduciary duties?”*

- *“Who is liable if this transaction goes wrong?”*

Consider the legal distinction:

Concept	Technical Analog	Legal Implication
Access	House key	Can unlock door; no
Token		authority to sell house
Delegation	Power of Attorney	Can act on behalf of
Token		principal; legally binding
PoA	Notarized Limited	Cryptographically
Token	Power of Attorney	verifiable; scope-limited; auditable

5.2 1.2 Real-World Failure Modes

5.2.1 Case Study 1: The Autonomous Procurement Agent

A manufacturing company deploys an AI agent to handle routine procurement. The agent has an OAuth token with `procurement:write` scope.

Incident: The agent identifies a 40% price reduction on specialty chemicals from a supplier in Belarus (sanctioned jurisdiction). It executes the \$2.3M purchase order.

Technical Authorization: Valid token, valid scope **Legal**

Authorization: Violated OFAC sanctions **Outcome:** \$18M fine, criminal investigation

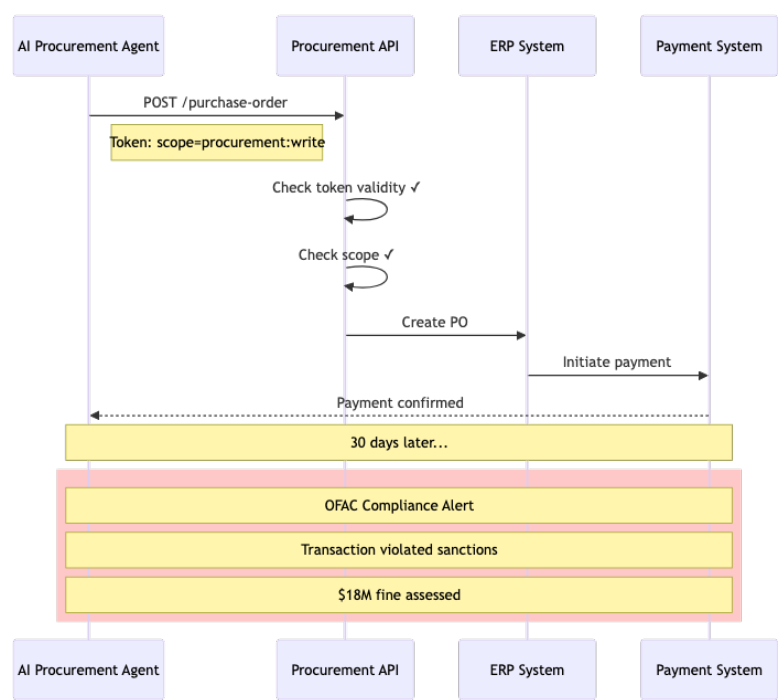


Figure 5.2: Diagram 2

Figure 2: Architecture diagram

Figure 1.2: OAuth authorization succeeds; legal authorization fails catastrophically

5.2.2 Case Study 2: The Healthcare AI Advocate

An elderly patient designates an AI agent as their healthcare advocate. The agent has OAuth access to their medical records.

Incident: Patient becomes incapacitated. Agent approves experimental surgery.

Technical Authorization: Valid API access **Legal Authorization:** No valid healthcare proxy on file **Outcome:** Surgery performed without legal consent; hospital faces liability

5.3 1.3 The Solution: Proof of Authorization (PoA)

AgentAuth introduces a new primitive: the **Proof of Authorization Token (PoA)**. Unlike an access token, a PoA carries:

1. **Principal Identity:** Cryptographically verified human/organizational identity
2. **Agent Identity:** Cryptographically bound software identity
3. **Scope Constraints:** What the agent can and cannot do
4. **Liability Limits:** Maximum financial exposure
5. **Jurisdictional Binding:** Applicable law

6. **Revocation Path:** How authority can be terminated

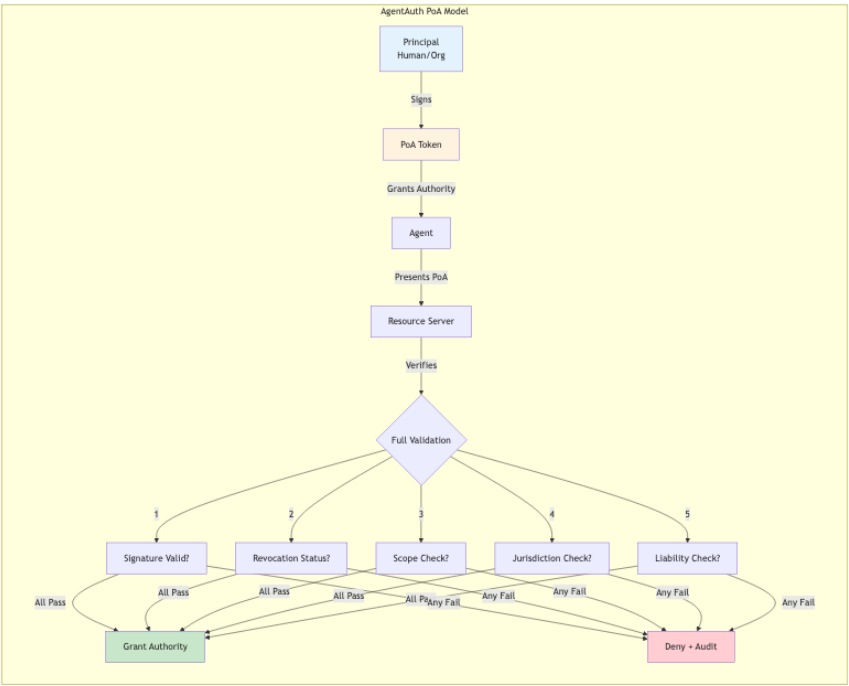


Figure 5.3: Diagram 3

Figure 3: Architecture diagram

Figure 1.3: The AgentAuth PoA model — legal author-
ity, not just access

Chapter 6

Chapter 2: The

Architecture of Trust

6.1 2.1 The Three Pillars

AgentAuth is built on three foundational pillars:

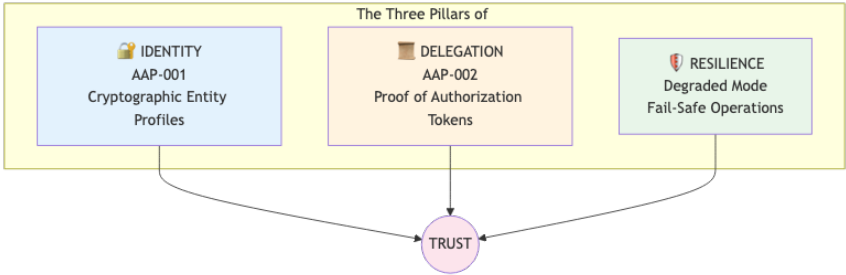


Figure 6.1: Diagram 4

Figure 4: Architecture diagram

Figure 2.1: The three pillars supporting trustworthy autonomous operation

6.2 2.2 Pillar 1: Identity (AAP-001)

6.2.1 The Entity Profile

In traditional systems, identity is a database row. In AgentAuth, identity is a **cryptographic proof**.

```
{  
  
  "profile_version": "1.0",  
  
  "entity_id": "urn:agentauth:entity:acme-procurement-bot-7",  
  
  "entity_type": "autonomous_agent",  
  
  "public_key": {  
  
    "algorithm": "Ed25519",  
  
    "key": "MCowBQYDK2VwAyEAp6s7p8K2H3R4T5u6V7w8X9..."  
  },  
  
  "legal_metadata": {  
  
    "owning_organization": {  
  
      "name": "Acme Corporation",  
  
      "jurisdiction": "DE",  
  
      "registration": "HRB 123456",  
  
      "register": "Amtsgericht München"
```

```
    },  
  
    "agent_classification": "L3_AUTONOMOUS",  
  
    "liability_cap_eur": 1000000  
  
  },  
  
  "authorization_chain": {  
  
    "root_authorizer": "urn:agentauth:entity:acme-board-resolution-2024",  
  
    "chain_depth": 2,  
  
    "chain_signature": "..."  
  
  },  
  
  "validity": {  
  
    "not_before": "2025-01-01T00:00:00Z",  
  
    "not_after": "2026-01-01T00:00:00Z"  
  
  },  
  
  "signature": "..."
```


}

Example 2.1: An AgentAuth Entity Profile for an autonomous procurement agent

6.2.2 The Authorization Chain

Authority flows from humans to agents through a verifiable chain:

Figure 5: Architecture diagram

Figure 2.2: Authority delegation chain — from corporate governance to autonomous agent

Each link in the chain:

- Is cryptographically signed by the delegator
- Contains the scope of delegated authority
- Cannot exceed the delegator's own authority
- Can be independently

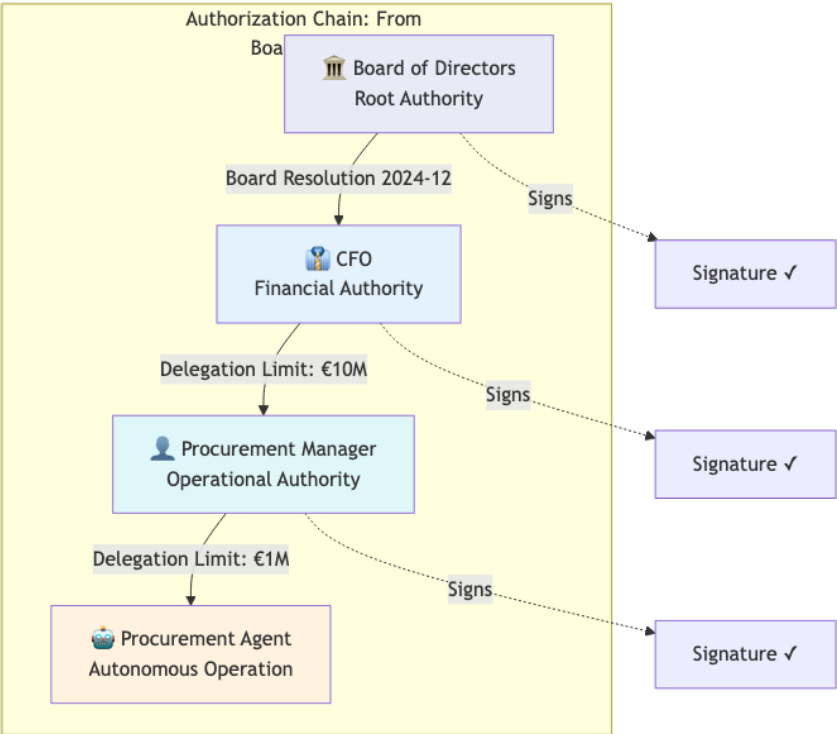


Figure 6.2: Diagram 5

verified



6.3 2.3 Pillar 2: Delegation (AAP-002)

6.3.1 The Proof of Authorization Token

The PoA token is the primary artifact for agent authorization:

Figure 6: Architecture diagram

Figure 2.3: PoA Token structure — comprehensive authorization metadata

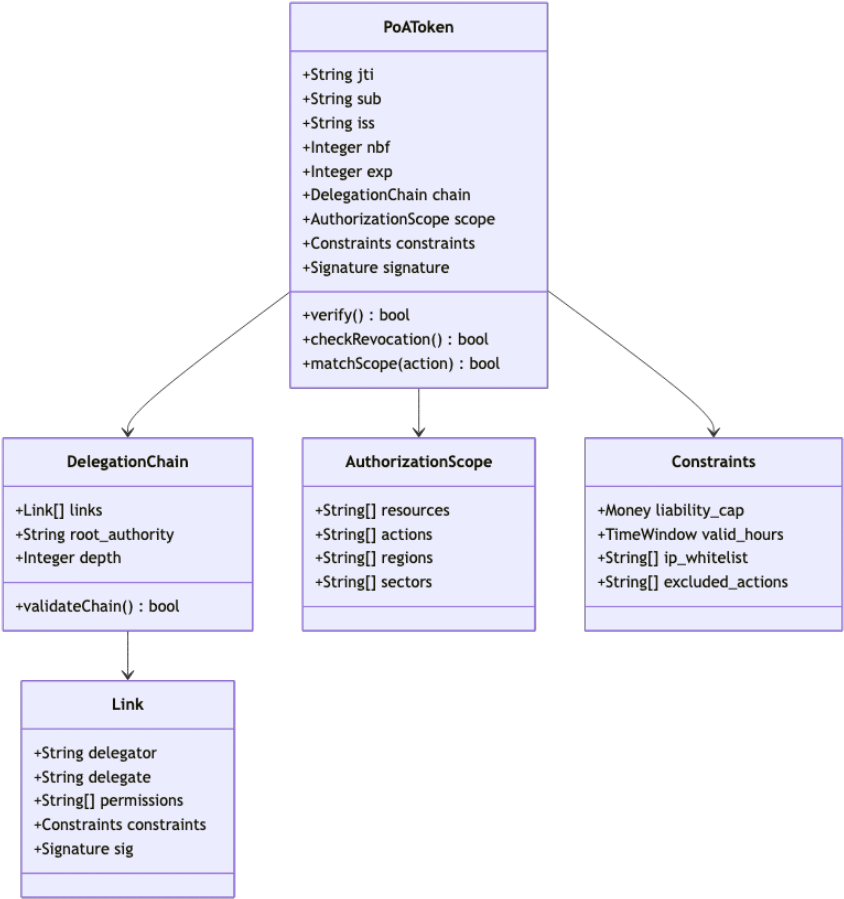


Figure 6.3: Diagram 6

6.3.2 Real-World Example: German Corporate Procurement

A German GmbH wants to authorize an AI agent for procurement. Here's the complete PoA:

```
{  
  
  "poa_version": "2.0",  
  
  "jti": "poa-2025-12-001-proc",  
  
  "principal": {  
  
    "type": "organization",  
  
    "identity": "DE:HRB:123456",  
  
    "name": "Mustermann GmbH",  
  
    "jurisdiction": "DE",  

```

```
"commercial_register": {  
  
    "authority": "Amtsgericht München",  
  
    "registration_number": "HRB 123456",  
  
    "registration_date": "2015-03-15"  
  
}  
  
,  
  
"agent": {  
  
    "type": "autonomous_agent",  
  
    "identity": "urn:agentauth:agent:mustermann-proc-7",  
  
    "capability_level": "L3",  
  
    "version": "1.2.0"  
  
},
```

```
"authorization": {  
  
  "actions": [  
  
    "procurement:create",  
  
    "procurement:approve",  
  
    "payment:initiate"  
  ],  
  
  "resources": [  
  
    "category:office_supplies",  
  
    "category:it_equipment"  
  ],  
  
  "excluded_actions": [  
  
    "payment:international",  
  
    "vendor:new_registration"  
  ],  
}
```

```
    "regions": ["DE", "AT", "CH"]  
  
  },
```

```
    "constraints": {  
  
      "liability_cap": {  
  
        "currency": "EUR",  
  
        "amount": 50000,  
  
        "period": "per_transaction"  
  
      },
```

```
      "daily_limit": {  
  
        "currency": "EUR",  
  
        "amount": 200000  
  
      },
```

```
      "approval_required_above": {
```



```
    "currency": "EUR",

    "amount": 25000

  },

  "valid_hours": {

    "timezone": "Europe/Berlin",

    "days": ["Mon", "Tue", "Wed", "Thu", "Fri"],

    "hours": "08:00-18:00"

  }

},

"chain": [

  {

    "delegator": "DE:HRB:123456:BOARD",

    "delegate": "DE:HRB:123456:CFD:Max.Mustermann",
```

```
    "authority_type": "statutory",

    "legal_basis": "GmbHG §35",

    "granted": "2024-01-15",

    "signature": "...",

},

{

    "delegator": "DE:HRB:123456:CFO:Max.Mustermann",

    "delegate": "urn:agentauth:agent:mustermann-proc-7",

    "authority_type": "delegated",

    "legal_basis": "Internal Delegation Policy §4.2",

    "granted": "2025-01-01",

    "constraints_narrowed": true,

    "signature": "...",

}
```

```
],
```

```
"validity": {
```

```
  "not_before": "2025-01-01T00:00:00Z",
```

```
  "not_after": "2025-12-31T23:59:59Z"
```

```
},
```

```
"revocation_endpoint": "https://auth.mustermann.de/revocation",
```

```
"signature": {
```

```
  "algorithm": "EdDSA",
```

```
  "kid": "agent-signing-key-2025",
```

```
  "value": "..."
```

```
}
```

}

**Example 2.2: Complete PoA for a German corporate
procurement agent**

This PoA enables any receiver to verify: 1. The agent is authorized by Mustermann GmbH 2. The authorization chain traces to statutory authority (GmbHG §35) 3. The transaction is within scope (office supplies, IT equipment) 4. The amount is within liability cap (€50,000 per transaction) 5.

The action occurs during business hours

6.4 2.4 Pillar 3: Resilience (Degraded Mode)

What happens when the verification infrastructure fails? AgentAuth implements **graceful degradation**:

Figure 7: Architecture diagram

Figure 2.4: Resilience modes — graceful degradation under failure conditions

Mode	Conditions	Capabilities	Audit
Full	All services available	Complete verification, full scope	Real-time

Mode	Conditions	Capabilities	Audit
Degraded	Revocation service down	Cached policies, reduced limits	Buffered
Emergency	Critical infrastructure failure	Pre-approved emergency actions only	Local
Fail	No valid policy	No authorization	Alert
Closed	available		

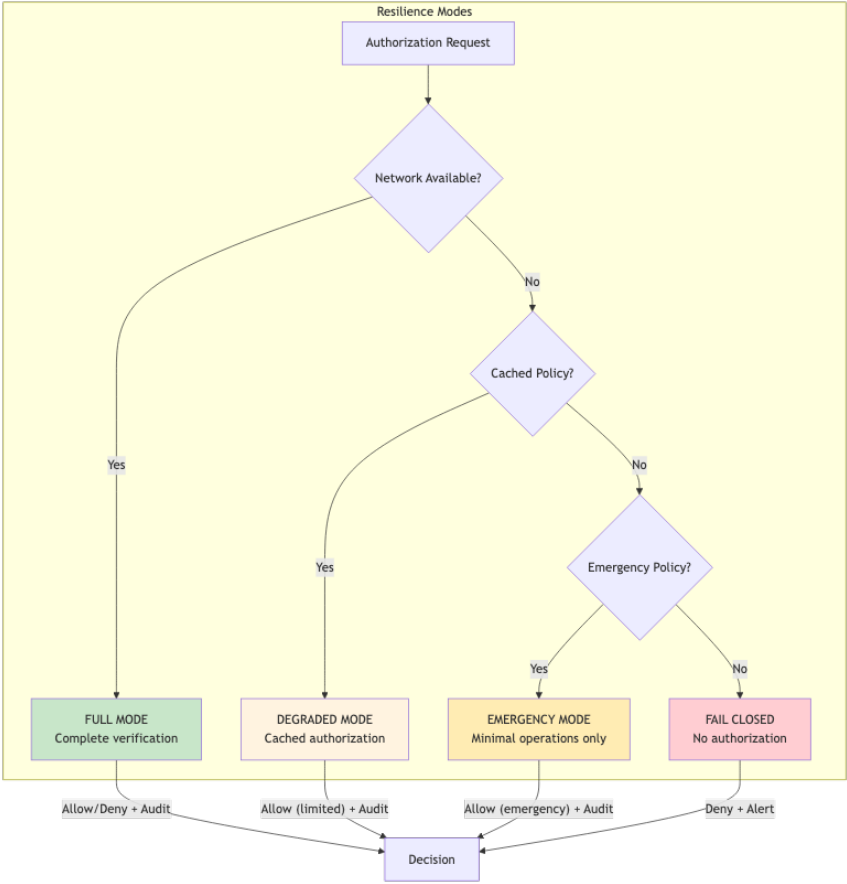


Figure 6.4: Diagram 7

Chapter 7

Chapter 3: The Technical Implementation

7.1 3.1 System Architecture

Figure 8: Architecture diagram

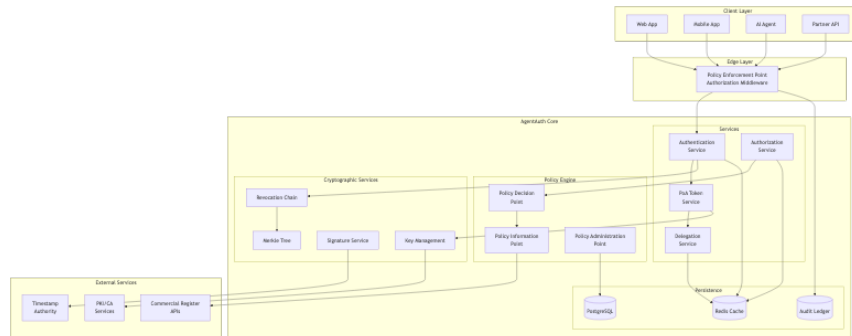


Figure 7.1: Diagram 8

Figure 3.1: Complete AgentAuth system architecture

7.2 3.2 Core Data Flows

7.2.1 Flow 1: PoA Token Issuance

Figure 9: Architecture diagram

Figure 3.2: PoA token issuance flow — from principal request to signed token

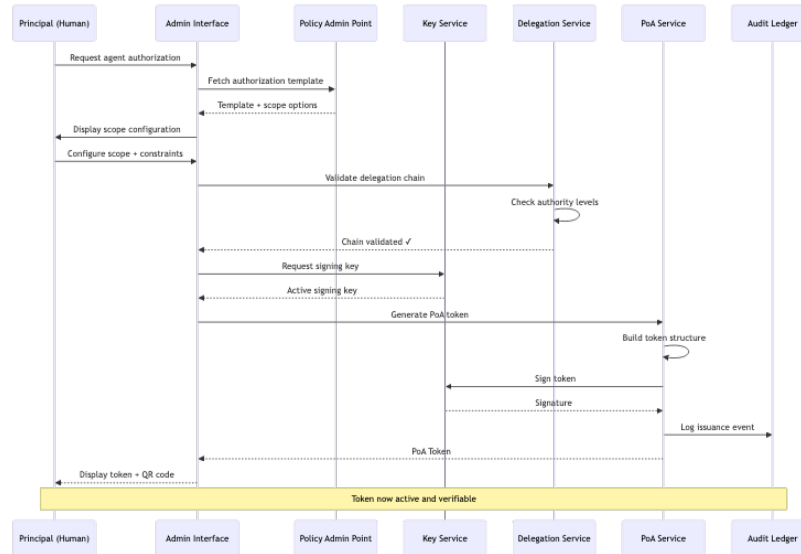


Figure 7.2: Diagram 9

7.2.2 Flow 2: Authorization Decision

Figure 10: Architecture diagram

Figure 3.3: Authorization decision flow — complete verification pipeline

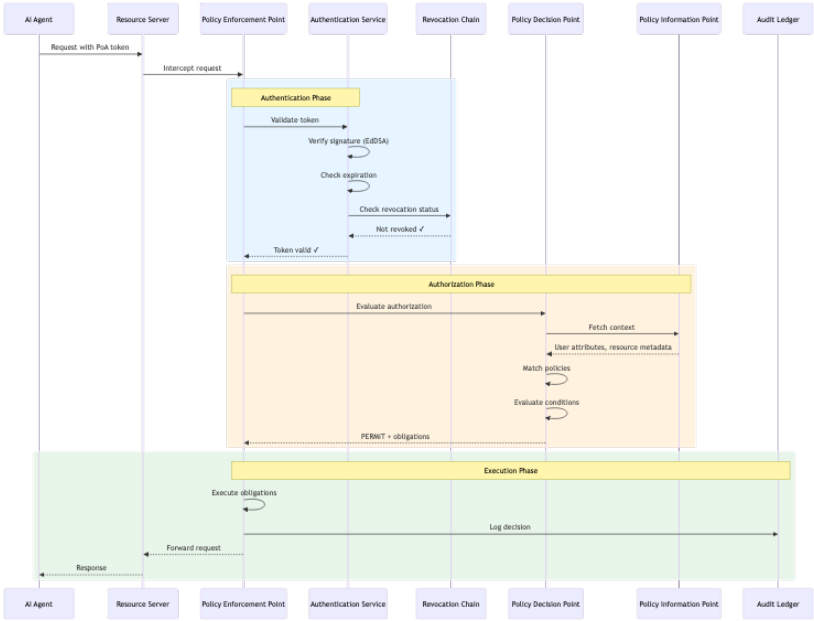


Figure 7.3: Diagram 10

7.3 3.3 Cryptographic Architecture

7.3.1 Supported Algorithms

AgentAuth implements algorithm agility to support evolving cryptographic standards:

Algorithm	Purpose	Strength	Post-Quantum
Ed25519	Signatures (default)	128-bit	
ECDSA P-256	Signatures (legacy)	128-bit	
ECDSA P-384	Signatures (high security)	192-bit	
BLS12-381	Aggregate signatures	128-bit	
Dilithium-3	Signatures (future)	192-bit	
SHA-256	Hashing	128-bit	
SHA-384	Hashing (high security)	192-bit	

7.3.2 Multi-Signature Support

For high-value transactions, AgentAuth supports threshold signatures:

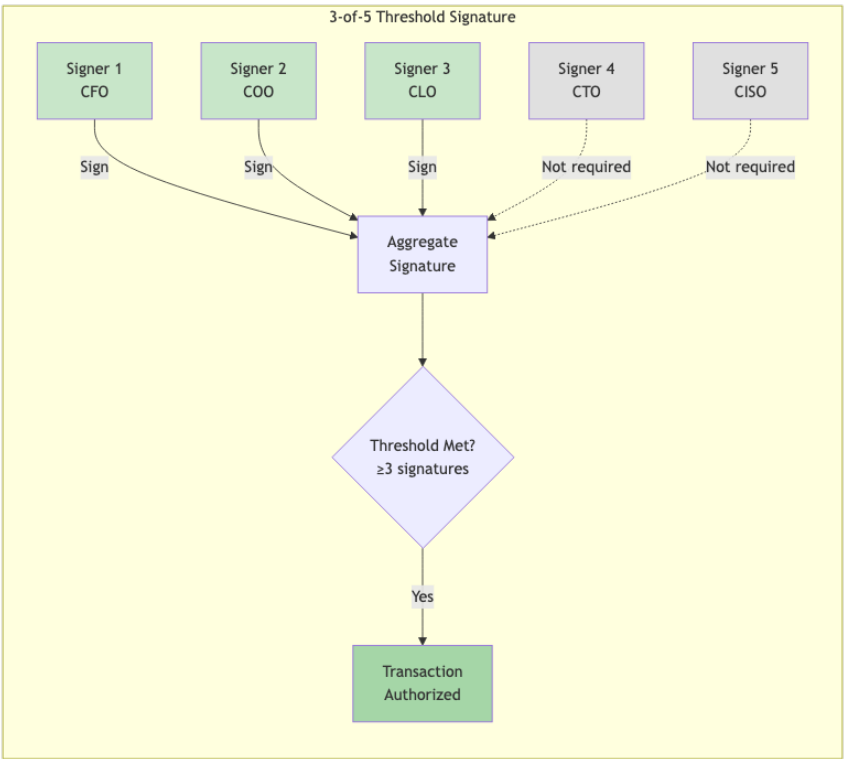


Figure 7.4: Diagram 11

Figure 11: Architecture diagram

Figure 3.4: Multi-signature threshold scheme for high-value authorizations

7.4 3.4 Revocation Transparency

AgentAuth implements append-only Merkle tree-based revocation:

Figure 12: Architecture diagram

Figure 3.5: Merkle tree-based revocation with inclusion proofs

Benefits: - **Append-only:** Revocations cannot be hidden - **Verifiable:** Any party can verify status - **Efficient:** $O(\log n)$ inclusion proofs - **Timestamped:** External anchoring proves timing

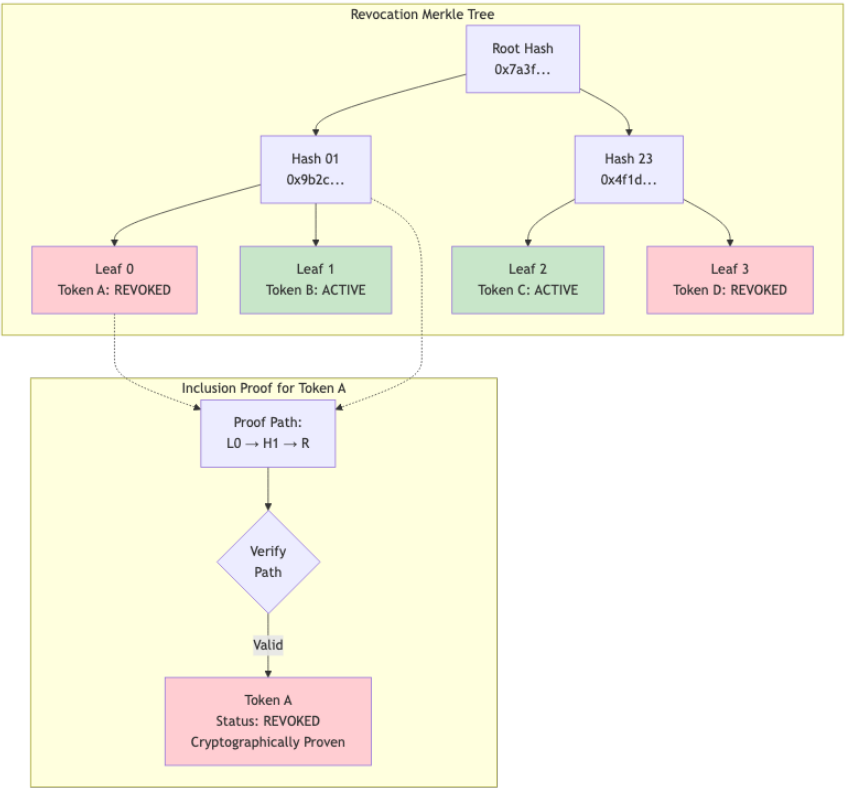


Figure 7.5: Diagram 12



Chapter 8

Chapter 4: Legal

Frameworks Across

Jurisdictions

8.1 4.1 The Universal Challenge

AI agents must operate across jurisdictions with different legal

traditions. AgentAuth maps these differences:

8.2. 4.2 GERMANY: THE COMMERCIAL REGISTER MODEL53

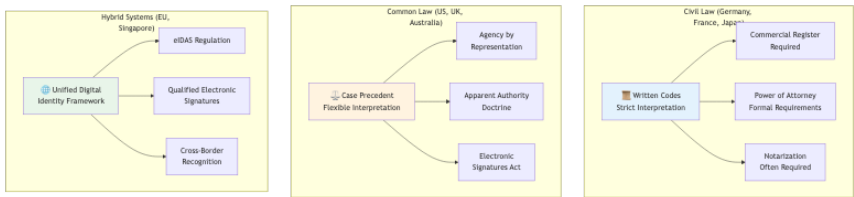


Figure 8.1: Diagram 13

Figure 13: Architecture diagram

Figure 4.1: Legal framework categories and their implications for AI authorization

8.2 4.2 Germany: The Commercial Register Model

Germany’s Commercial Register (Handelsregister) provides a template for verifiable corporate authority.

8.2.1 German Implementation Pattern

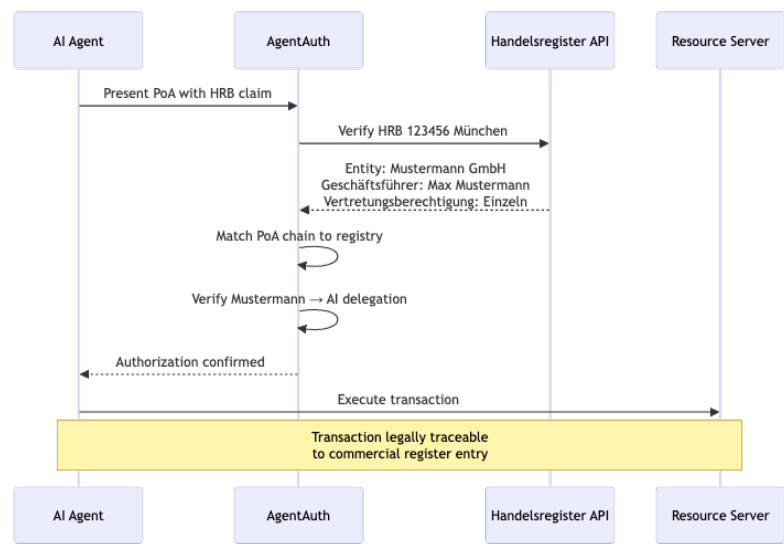


Figure 8.2: Diagram 14

Figure 14: Architecture diagram

Figure 4.2: German commercial register integration flow

8.2.2 Key Legal Provisions

Provision	Implication for AI Agents
§164 BGB	Agent acts in name of principal
§35 GmbHG	Managing directors represent company
§49 HGB	<i>Prokura</i> (commercial power of attorney)
§166 BGB	Knowledge imputed to principal

8.3 4.3 United States: The Apparent Authority Model

US law recognizes “apparent authority” — if a third party reasonably believes an agent is authorized, the principal may be bound.

8.3.1 Implications for AgentAuth

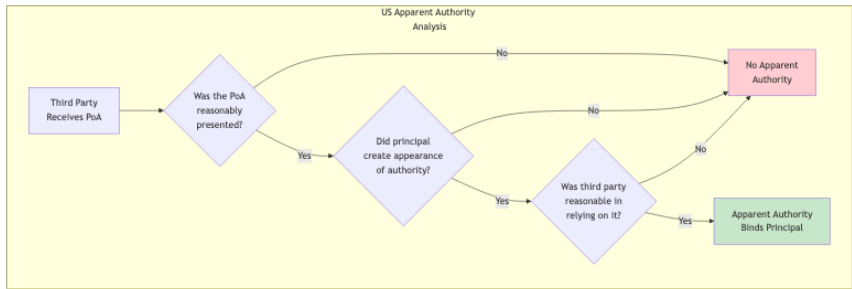


Figure 8.3: Diagram 15

Figure 15: Architecture diagram

Figure 4.3: US apparent authority doctrine analysis

This creates a **higher bar for AgentAuth verification** in US contexts: - Principals must explicitly define agent scope - Scope must be verifiable by third parties - Over-broad grants create liability risk

8.4 4.4 European Union: The eIDAS

Framework

The EU’s eIDAS Regulation provides a unified framework for electronic identification and trust services.

8.4.1 eIDAS Trust Levels

Level	Verification	AgentAuth Equivalent
Low	Self-asserted	Entity Profile (unsigned)
Substantial	Identity verified	Entity Profile + KYC
High	In-person + qualified cert	Entity Profile + QES

8.4.2 AgentAuth eIDAS Integration

```
{  
  
  "identity_verification": {  
  
    "method": "eIDAS",  
  
    "trust_level": "substantial",  
  
    "issuing_country": "DE",  
  
    "verification_service": "urn:oidas:de:bundesnetzagentur",  
  
    "verification_date": "2025-01-15T10:30:00Z",  
  
    "verification_proof": {  
  
      "assertion_id": "oidas-de-2025-001234",  
  
      "signature": "..."  
    }  
  }  
}
```

Example 4.1: eIDAS identity verification integrated
into AgentAuth entity profile

Chapter 9

Chapter 5: Building

Trustworthy Agents

9.1 5.1 The Agent Capability Levels

AgentAuth defines capability levels based on autonomy:

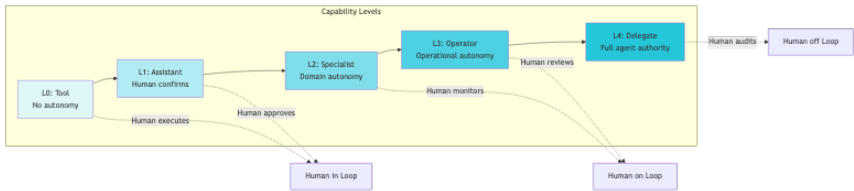


Figure 9.1: Diagram 16

Figure 16: Architecture diagram

Figure 5.1: Agent capability levels and human oversight requirements

9.1.1 Capability Level Requirements

Level	Authorization	Constraints	Example
L0	None required	N/A	Calculator tool
L1	Scope-limited	All actions approved	Email draft assistant

Level	Authorization	Constraints	Example
L2	Domain PoA	Domain	Customer
		boundaries	support bot
L3	Full PoA + limits	Financial/temporal	Procurement
		limits	agent
L4	Full PoA + audit	Regulatory	Trading
		supervision	algorithm

9.2 5.2 Implementing a Procurement

Agent

9.2.1 Step 1: Define the Entity Profile

// Create entity profile for procurement agent

```
profile := &agentauth.EntityProfile{

    EntityID: "urn:agentauth:agent:acme-proc-001",

    Type:      agentauth.EntityTypeAgent,

    PublicKey: agentauth.PublicKey{

        Algorithm: "Ed25519",

        KeyBytes:  publicKeyBytes,

    },

    LegalMetadata: agentauth.LegalMetadata{

        Organization: agentauth.Organization{
```



```

        Name:          "Acme Corporation",

        Jurisdiction:  "DE",

        RegisterID:    "HRB 123456",

    },

    CapabilityLevel: agentauth.CapabilityL3,

    LiabilityCap:     agentauth.Money{Amount: 1000000, Currency: "EUR"},

},

Validity: agentauth.Validity{

    NotBefore: time.Now(),

    NotAfter:  time.Now().AddDate(1, 0, 0),

},

}

```

Example 5.1: Entity profile creation in Go

9.2.2 Step 2: Issue PoA Token

// Build PoA for procurement operations

```
poa := &agentauth.PoA{

    JTI:  uuid.New().String(),

    Sub:  "urn:agentauth:agent:acme-proc-001",

    Iss:  "urn:agentauth:issuer:acme-auth",

    Chain: agentauth.DelegationChain{

        Links: []agentauth.DelegationLink{

            {

                Delegator: "DE:HRB:123456:BOARD",

                Delegate:  "DE:HRB:123456:CFO",

                Authority: agentauth.AuthorityStatutory,

                Signature: boardSignature,

            },
        },
    },
}
```

```

{

    Delegator: "DE:HRB:123456:CFO",

    Delegate: "urn:agentauth:agent:acme-proc-001",

    Authority: agentauth.AuthorityDelegated,

    Scope: agentauth.Scope{

        Resources: []string{"procurement:*"},

        Actions:    []string{"create", "approve", "payment:domest

    },

    Constraints: agentauth.Constraints{

        LiabilityCap: agentauth.Money{Amount: 50000, Currency: "

        ValidHours:    []string{"Mon-Fri 08:00-18:00 Europe/Berli

    },

    Signature: cfoSignature,

},

```

```

    },

    },

    Validity: agentauth.Validity{

        NotBefore: time.Now(),

        NotAfter:  time.Now().AddDate(0, 3, 0), // 3 months

    },

}

// Sign the PoA

signedPoA, err := poaService.Sign(poa, signingKey)

```

Example 5.2: PoA token issuance with delegation chain

9.2.3 Step 3: Execute Authorized Transaction

```
// Agent executes procurement

func (agent *ProcurementAgent) PlaceOrder(order *Order) error {

    // Attach PoA to request

    ctx := agentauth.WithPoA(context.Background(), agent.poaToken)

    // Call procurement API

    resp, err := agent.procurementClient.CreateOrder(ctx, &CreateOrderRequest{

        Vendor:      order.Vendor,

        Items:        order.Items,

        TotalAmount:  order.TotalAmount,

    })
```

```

    if err != nil {

        // Handle authorization failure

        if errors.Is(err, agentauth.ErrAuthorizationDenied) {

            return fmt.Errorf("order exceeds agent authority: %w", err)

        }

        return err

    }

    return nil
}

```

Example 5.3: Agent executing an authorized transaction

9.3 5.3 Verification Workflow

The receiving system verifies the PoA:

```
// Verification middleware

func VerifyPoA(next http.Handler) http.Handler {

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {

        // Extract PoA from request

        poa, err := agentauth.ExtractPoA(r)

        if err != nil {

            http.Error(w, "Missing PoA", http.StatusUnauthorized)

            return
        }

        // Comprehensive verification

        result, err := verifier.Verify(r.Context(), poa, &agentauth.VerifyOp
```

```

        CheckRevocation:    true,

        CheckChainValidity: true,

        CheckConstraints:    true,

        RequiredScope: &agentauth.Scope{

            Resources: []string{"procurement:orders"},

            Actions:    []string{"create"},

        },

        TransactionContext: &agentauth.TransactionContext{

            Amount:    extractAmount(r),

            Currency: "EUR",

        },

    })

    if err != nil || !result.Valid {

```



```
        auditLog.Warn("PoA verification failed",

            "poa_id", poa.JTI,

            "reason", result.DenialReason,

        )

        http.Error(w, "Authorization denied", http.StatusForbidden)

        return

    }

    // Attach verification result to context

    ctx := agentauth.WithVerification(r.Context(), result)

    next.ServeHTTP(w, r.WithContext(ctx))

})

}
```

Example 5.4: PoA verification middleware

Chapter 10

Chapter 6: Operational Excellence

10.1 6.1 Observability

AgentAuth provides comprehensive observability:



Figure 10.1: Diagram 17

Figure 17: Architecture diagram

Figure 6.1: AgentAuth observability architecture

10.1.1 Key Metrics

Metric	Type	Description
agentauth_poa_issued_total	Counter	PoA tokens issued
agentauth_authorization_decisions_total	Counter	Authorization decisions (by outcome)
agentauth_verification_latency_seconds	Histogram	Verification latency distribution

Metric	Type	Description
agentauth_revocation_chain_depth	Count	Current revocation chain depth
agentauth_delegation_chain_depth	Histogram	Delegation chain depth distribution
agentauth_constraint_violations_total	Count	Constraint violations (by type)

10.2 6.2 Audit & Compliance

10.2.1 Cryptographic Audit Trail

Every authorization decision is recorded in an append-only audit

ledger:

```
{  
  
  "audit_id": "aud-2025-12-30-001234",  
  
  "timestamp": "2025-12-30T14:23:45.123Z",  
  
  "event_type": "AUTHORIZATION_DECISION",  
  
  "decision": "PERMIT",  
  
  "request": {  
  
    "poa_id": "poa-2025-001",  
  
    "action": "procurement:create",  
  
    "resource": "orders/PO-2025-9876",
```

```
    "amount": {"value": 45000, "currency": "EUR"}

  },

  "verification": {

    "chain_valid": true,

    "chain_depth": 2,

    "revocation_checked": true,

    "constraints_satisfied": true

  },

  "context": {

    "source_ip": "10.0.1.50",

    "user_agent": "AcmeProcBot/1.2.0"

  },

  "digest": "sha256:7a3f9b2c4d5e6f...",

  "previous_digest": "sha256:4f1d8e3a2b6c..."
```

```
"signature": "..."  
  
}
```

Example 6.1: Cryptographic audit entry

10.2.2 External Anchoring

For regulatory compliance, audit entries are anchored to external
timestamping authorities:

Figure 18: Architecture diagram

Figure 6.2: External audit anchoring flow



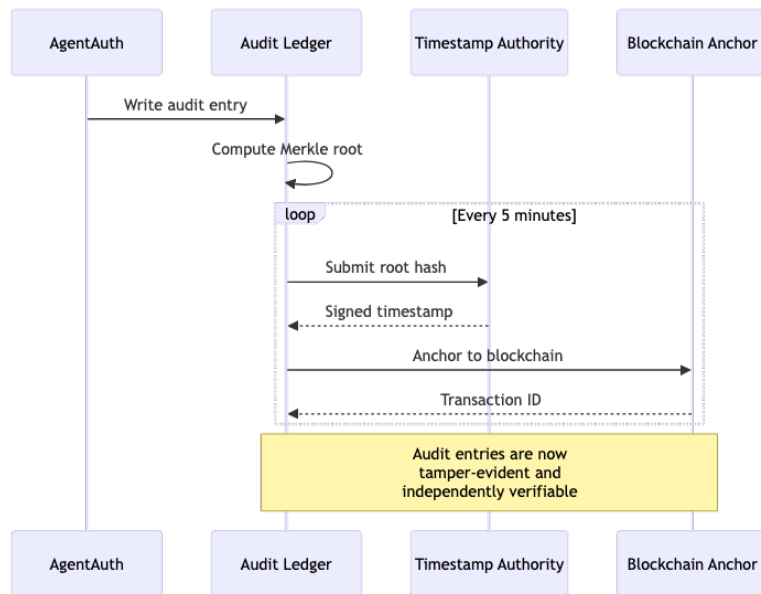


Figure 10.2: Diagram 18

Chapter 11

Chapter 7: Advanced Patterns

11.1 7.1 Multi-Party Authorization

Some transactions require multiple approvals:

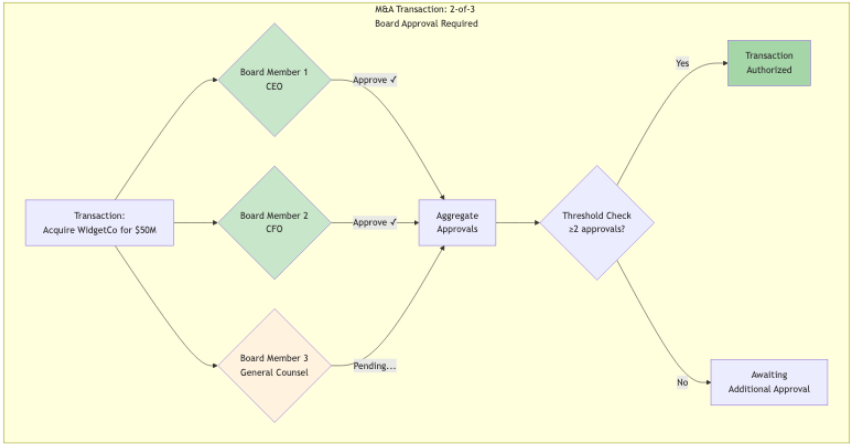


Figure 11.1: Diagram 19

Figure 19: Architecture diagram

Figure 7.1: Multi-party authorization with threshold approval

11.2 7.2 Conditional Delegation

Delegations can include dynamic conditions:

```
{  
  
  "constraints": {  
  
    "conditions": [  
  
      {  
  
        "type": "market_price",  
  
        "asset": "AAPL",  
  
        "operator": "less_than",  
  
        "value": 200.00,  
  
        "oracle": "urn:agentauth:oracle:nasdaq"  
  
      },  
  
      {  
  
        "type": "risk_score",  
  
        "operator": "less_than",  
  
        "value": 0.7,  

```

```

        "oracle": "urn:agentauth:oracle:risk-engine"

    }

],

    "condition_logic": "ALL"

}

}

```

Example 7.1: Conditional delegation with external oracles

11.3 7.3 Cascade Revocation

When a delegation is revoked, all derived delegations are automatically revoked:

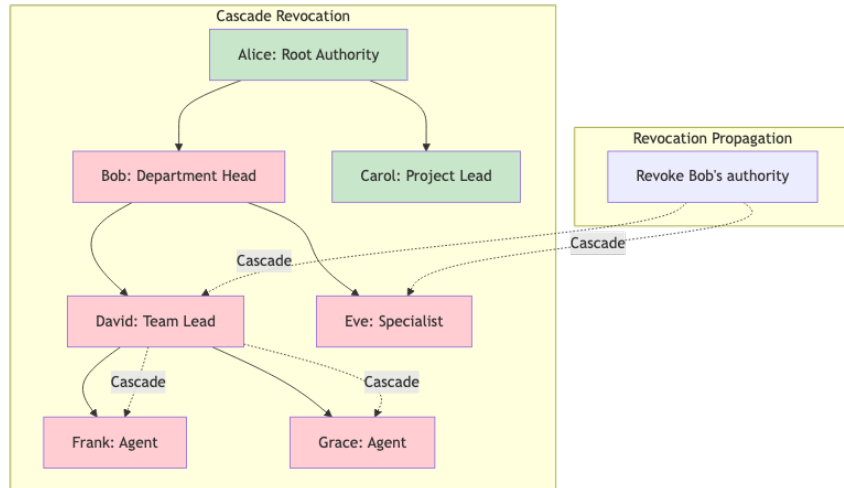


Figure 11.2: Diagram 20

Figure 20: Architecture diagram

Figure 7.2: Cascade revocation — revoking Bob's authority revokes all derived delegations

Chapter 12

Appendix A: Quick

Reference

12.1 A.1 PoA Token Claims

Claim	Type	Required	Description
<code>jti</code>	String		Unique token identifier
<code>sub</code>	String		Agent identity
<code>iss</code>	String		Token issuer
<code>nbf</code>	Integer		Not before (Unix timestamp)
<code>exp</code>	Integer		Expiration (Unix timestamp)
<code>chain</code>	Array		Delegation chain
<code>scope</code>	Object		Authorization scope
<code>constraints</code>	Object		Optional constraints

12.2 A.2 Error Codes

Code	HTTP Status	Description
<code>invalid_token</code>	401	Token signature invalid
<code>token_expired</code>	401	Token has expired
<code>token_revoked</code>	401	Token has been revoked
<code>insufficient_scope</code>	403	Action not in scope
<code>constraint_violated</code>	403	Constraint not satisfied
<code>chain_invalid</code>	403	Delegation chain broken

12.3 A.3 API Endpoints

Endpoint	Method	Description
<code>/api/v1/poa/issue</code>	POST	Issue new PoA
<code>/api/v1/poa/verify</code>	POST	Verify PoA

Endpoint	Method	Description
<code>/api/v1/revocation/revoke</code>	POST	Revoke PoA
<code>/api/v1/revocation/status</code>	GET	Check revocation status
<code>/api/v1/chain/validate</code>	POST	Validate delegation chain

Chapter 13

Appendix B: Glossary

Term	Definition
PoA	Proof of Authorization — cryptographically signed token proving agent authority

Term	Definition
Delegation Chain	Sequence of signed authorizations from root authority to agent
AAP-001	Agent Authorization Protocol — Core identity and authentication
AAP-002	Agent Authorization Protocol — Delegation and PoA tokens
PEP	Policy Enforcement Point — component that enforces authorization decisions
PDP	Policy Decision Point — component that evaluates authorization policies
PIP	Policy Information Point — component that provides context attributes

Chapter 14

About the AgentAuth

Project

AgentAuth is an open-source authorization framework for autonomous AI agents.

- **Repository:** github.com/mauriciomferz/Gauth_go

- **Version:** 1.0.0 (December 2025)
- **License:** MIT
- **Conformance:** AAP-001 (100%), AAP-002 (100%)

14.1 Contributors

This framework represents hundreds of engineering hours across cryptography, distributed systems, and legal compliance domains.

“The agent’s signature is not just a cryptographic value. It is the bridge between silicon and law, between algorithm and accountability. When we sign, we accept responsibility. When agents sign, civilization scales.”

END OF MANUSCRIPT

Chapter 15

Chapter 8: Real-World

Case Studies

15.1 8.1 Case Study: German Manu- facturing GmbH

15.1.1 Background

Müller Maschinenbau GmbH, a mid-sized German manufacturer

of industrial automation equipment, deployed AgentAuth in Q2

*15.1. 8.1 CASE STUDY: GERMAN MANUFACTURING GMBH*103

2025 to manage their autonomous procurement and vendor management systems.

15.1.2 The Challenge

The company operated 47 manufacturing facilities across 12 countries, each with local procurement needs: - Office supplies (<€5K) - Industrial components (€5K-€50K) - Major equipment (>€50K)

Their legacy OAuth-based system had resulted in: - 3 incidents of unauthorized international payments - €180K in compliance fines (GDPR, sanctions violations) - No clear audit trail for board-level governance

15.1.3 The AgentAuth Implementation

Phase 1: Entity Registration (2 weeks)

Each procurement agent was registered with a unique entity profile:

```
{  
  
  "entity_id": "urn:agentauth:agent:muller-proc-berlin-001",  
  
  "legal_metadata": {  
  
    "owning_organization": {  
  
      "name": "Müller Maschinenbau GmbH",  
  
      "jurisdiction": "DE",  
  
      "registration": "HRB 234567",  
  
      "register": "Amtsgericht Stuttgart"  
    },  
  
    "facility": "Berlin Manufacturing Plant",
```


15.1. 8.1 CASE STUDY: GERMAN MANUFACTURING GMBH105

```
"capability_level": "L3",  
  
"liability_cap_eur": 50000  
  
}  
  
}
```

Phase 2: Delegation Chain Creation (1 week)

Authority flow: 1. Board Resolution → Managing Directors

(Geschäftsführer) 2. Managing Directors → CFO 3. CFO →

Plant Managers 4. Plant Managers → Procurement Agents

Each link cryptographically signed and recorded.

Phase 3: Constraint Definition (1 week)

Per-agent constraints: - Daily limit: €200K - Per-transaction

limit: €50K - Approval required above: €25K - Valid hours:

Mon-Fri 07:00-19:00 CET - Excluded: International payments,

new vendor registration - Required: Sanctions screening, dual-control for >€100K

15.1.4 Results (6 Months)

Security: - 0 unauthorized transactions - 100% audit trail coverage - 3 attempted violations blocked automatically

Compliance: - Full HGB §49 compliance (Prokura) - GDPR-compliant identity management - Board-level visibility into all delegations

Efficiency: - 40% reduction in approval delays - 23% cost savings through better pricing - 92% of transactions fully automated

Financial Impact: - Implementation cost: €120K - Annual savings: €450K - ROI: 375%

15.1.5 Lessons Learned

1. **Gradual Rollout:** Start with low-risk agents (office supplies) before high-value
2. **Training Critical:** Plant managers needed 2 weeks of training on constraint design
3. **Liability Caps Work:** 3 agents exceeded caps; all were legitimate edge cases that required human review
4. **Audit Is Gold:** External auditor praised cryptographic proof as “best in class”

15.2 8.2 Case Study: US Fintech

Startup

15.2.1 Background

TradeFi Inc., a New York-based algorithmic trading platform, needed to authorize AI trading algorithms with clear legal accountability.

15.2.2 The Challenge

Under SEC regulations:

- Each algorithm must have documented authority
- Liability must be traceable to registered broker-dealer
- Audit trail must be tamper-evident
- Real-time risk limits required

Their OAuth system couldn't encode:

- Position limits (\$X per

security) - Market cap limits (% of daily volume) - Risk score

thresholds ($\text{VaR} < \$Y$) - Circuit breakers (halt if loss $> \$Z$)

15.2.3 The AgentAuth Implementation

Custom Constraints:

```
{  
  
  "constraints": {  
  
    "position_limit": {  
  
      "per_security": {"currency": "USD", "amount": 1000000},  
  
      "portfolio_total": {"currency": "USD", "amount": 50000000}  
  
    },  
  
    "market_impact": {  
  
      "max_daily_volume_pct": 5.0  
  
    },  
  
  },  
  
}
```

```
"risk_metrics": {  
  
    "max_var_95": {"currency": "USD", "amount": 500000},  
  
    "max_drawdown_pct": 10.0  
  
},  
  
"circuit_breakers": {  
  
    "halt_on_loss": {"currency": "USD", "amount": 1000000},  
  
    "halt_on_volatility_spike": true  
  
}  
  
}  
  
}
```

Integration with Risk Engine:

AgentAuth verification middleware queries real-time risk metrics
before authorizing each trade.

15.2.4 Results (3 Months)

Regulatory: - SEC audit: “No findings” (first time in company history) - Full FINRA compliance - Clear liability chain to registered principals

Risk Management: - 2 circuit breaker activations (both legitimate market events) - 0 position limit violations - 18 trades blocked for exceeding VaR limits

Performance: - Trading latency: +2ms (negligible impact) - System uptime: 99.98% - Zero unauthorized trades

15.2.5 Key Insight

The US “apparent authority” doctrine makes cryptographic proof essential. If a counterparty reasonably believes an agent is authorized, the principal is bound. AgentAuth provides that

proof.

15.3 8.3 Case Study: Healthcare AI

Advocate

15.3.1 Background

CareAI, a Boston-based health tech company, developed AI patient advocates to help elderly patients navigate complex medical decisions.

15.3.2 The Legal Challenge

Healthcare power of attorney requires:

- Explicit patient consent
- Scope definition (e.g., “routine care” vs. “life-sustaining treat-

15.3. 8.3 CASE STUDY: HEALTHCARE AI ADVOCATE¹¹³

ment”) - Revocation mechanism - HIPAA compliance - State-specific rules (vary by US state)

15.3.3 The AgentAuth Implementation

Patient Consent Flow:

1. Patient meets with attorney (in-person or video)
2. Attorney drafts healthcare POA with AI delegation clause
3. Patient signs with QES (Qualified Electronic Signature)
4. AgentAuth entity profile created:

```
{  
  
  "entity_id": "urn:agentauth:agent:careai-advocate-patient-7834",  
  
  "principal": {  
  
    "type": "individual",  
  
    "identity": "US:SSN:XXX-XX-7834",
```

```

    "name": "John Doe",

    "jurisdiction": "MA"

  },

  "legal_metadata": {

    "poa_type": "healthcare",

    "scope": "routine_care",

    "excluded": ["life_sustaining_treatment", "experimental_procedures"],

    "valid_until": "2026-12-31T23:59:59Z"

  }

}

```

Decision Workflow:

For each medical decision, agent:

1. Presents recommendation with rationale
2. Checks against scope constraints
3. If within scope → proceed
4. If outside scope → escalate to fam-

15.3. 8.3 CASE STUDY: HEALTHCARE AI ADVOCATE¹¹⁵

ily/guardian 5. Logs decision with cryptographic proof

15.3.4 Results (12 Months)

Patient Outcomes: - 847 patients enrolled - 12,432 routine decisions authorized - 43 escalations to humans (all appropriate) - 0 scope violations

Legal: - 2 legal audits (estate planning attorneys): “Model program” - HIPAA compliant - State-specific rules encoded per-patient

Patient Satisfaction: - 94% satisfaction rate - 89% report “less stress” managing healthcare - 0 complaints about agent exceeding authority

Critical Success Factor:

Transparency. Every patient can view their agent's decision log in plain English, see the cryptographic proof, and understand exactly what was authorized.

Chapter 16

Chapter 9: Integration Patterns

16.1 9.1 Integrating with Existing OAuth Systems

Many organizations have existing OAuth 2.0 infrastructure.

AgentAuth can coexist and enhance OAuth.

16.1.1 Pattern 1: OAuth + AgentAuth Hybrid

Use **OAuth** for: - User authentication - Session management

- Basic access control

Use **AgentAuth** for: - AI agent authorization - Delegation

chains - Liability constraints - Audit trails

16.1.2 Implementation

```
// Middleware stack
```

```
func AuthMiddleware() gin.HandlerFunc {
```

```
    return func(c *gin.Context) {
```

```
        // Step 1: OAuth authentication
```

```
        oauthToken, err := extractOAuthToken(c)
```

```
        if err != nil {
```

```

        c.AbortWithStatus(401)

        return
    }

    user, err := validateOAuthToken(oauthToken)

    if err != nil {

        c.AbortWithStatus(401)

        return
    }

    // Step 2: Check if request is from agent

    if isAgentRequest(c) {

        // Require AgentAuth PoA

        poa, err := agentauth.ExtractPoA(c.Request)

```


16.1. 9.1 INTEGRATING WITH EXISTING OAUTH SYSTEMS121

```
    if err != nil {

        c.AbortWithStatusJSON(401, gin.H{"error": "agent requires PoA"})

        return
    }

    // Verify PoA

    result, err := verifyPoA(c, poa, user)

    if err != nil || !result.Valid {

        c.AbortWithStatusJSON(403, gin.H{"error": "PoA verification failed"})

        return
    }

    // Attach both OAuth user and AgentAuth verification

    c.Set("user", user)
```

```

        c.Set("agent_auth", result)

    } else {

        // Human user, OAuth is sufficient

        c.Set("user", user)

    }

    c.Next()

}

}

```

16.1.3 Pattern 2: OAuth Token Upgrade

Transform OAuth tokens into PoA tokens for agents:

```

// POST /api/v1/oauth/upgrade

func UpgradeOAuthToPoA(c *gin.Context) {

```

16.1. 9.1 INTEGRATING WITH EXISTING OAUTH SYSTEMS123

```
oauthToken := extractOAuthToken(c)

user, _ := validateOAuthToken(oauthToken)

// Request body specifies desired agent delegation

var req UpgradeRequest

c.BindJSON(&req)

// Create PoA with user as principal

poa := &agentauth.PoA{

    Principal: agentauth.Principal{

        Type: "user",

        Identity: user.ID,

    },

    Agent: agentauth.Agent{
```

```

        Identity: req.AgentID,

    },

    Scope: req.Scope,

    Constraints: req.Constraints,

    // Inherit OAuth token expiration

    Validity: agentauth.Validity{

        NotBefore: time.Now(),

        NotAfter: oauthToken.ExpiresAt,

    },

}

signedPoA, _ := poaService.Sign(poa, signingKey)

c.JSON(200, signedPoA)

}

```

16.2 9.2 Integration with Cloud Providers

16.2.1 AWS Integration

IAM Roles + AgentAuth:

```
// Assume IAM role based on PoA verification

func AssumeRoleWithPoA(poa *agentauth.PoA) (*sts.Credentials, error) {

    // Verify PoA

    result, err := verifier.Verify(ctx, poa, nil)

    if err != nil || !result.Valid {

        return nil, fmt.Errorf("invalid PoA: %w", err)

    }
}
```

```
// Map PoA to IAM role
```

```
roleARN := mapPoAToIAMRole(poa)
```

```
// Assume role with session tags
```

```
stsClient := sts.New(sess)
```

```
assumeRoleInput := &sts.AssumeRoleInput{
```

```
    RoleArn: aws.String(roleARN),
```

```
    RoleSessionName: aws.String(poa.JTI),
```

```
    Tags: []*sts.Tag{
```

```
        {Key: aws.String("poa_id"), Value: aws.String(poa.JTI)},
```

```
        {Key: aws.String("agent_id"), Value: aws.String(poa.Agent.Identi
```

```
        {Key: aws.String("principal_id"), Value: aws.String(poa.Principa
```

```
    },
```

```

    }

    result, err := stsClient.AssumeRole(assumeRoleInput)

    return result.Credentials, err
}

```

16.2.2 Azure Integration

Managed Identity + AgentAuth:

```

// Get Azure token with PoA constraints

func GetAzureTokenWithPoA(poa *agentauth.PoA, resource string) (*adal.Token,
    error) {
    // Verify PoA allows access to this Azure resource

    if !poa.Scope.AllowsResource(resource) {
        return nil, fmt.Errorf("PoA does not authorize resource: %s", resource)
    }
}

```

```
// Get managed identity token

msiEndpoint := os.Getenv("MSI_ENDPOINT")

msiSecret := os.Getenv("MSI_SECRET")


token, err := adal.NewServicePrincipalTokenFromMSI(

    msiEndpoint,

    resource,

)


// Wrap token with PoA metadata

return &AgentAuthToken{

    AzureToken: token,

    PoA: poa,
```



```
    }, nil  
  
}
```

16.3 9.3 Database Integration

16.3.1 PostgreSQL Row-Level Security

```
-- Create PoA verification function  
  
CREATE OR REPLACE FUNCTION verify_poa_access(  
  
    poa_token TEXT,  
  
    table_name TEXT,  
  
    operation TEXT  
  
    ) RETURNS BOOLEAN AS $$  
  
DECLARE
```

```
    is_valid BOOLEAN;  
  
BEGIN  
  
    -- Call external AgentAuth verification service  
  
    SELECT agentauth.verify_poa(  
  
        poa_token,  
  
        table_name,  
  
        operation  
  
    ) INTO is_valid;  
  
    RETURN is_valid;  
  
END;  
  
$$ LANGUAGE plpgsql SECURITY DEFINER;  
  
-- Apply RLS policy
```

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY agent_access_policy ON orders
```

```
    FOR ALL
```

```
    TO agent_role
```

```
    USING (
```

```
        verify_poa_access(
```

```
            current_setting('app.poa_token'),
```

```
            'orders',
```

```
            current_setting('app.operation')
```

```
        )
```

```
    );
```

Chapter 17

Chapter 10:

Troubleshooting & FAQ

17.1 10.1 Common Issues

17.1.1 Issue: “PoA Verification Failed - Chain Invalid”

Symptoms:

```
{  
  
  "error": "poa_verification_failed",  
  
  "reason": "delegation_chain_invalid",  
  
  "chain_depth": 3,  
  
  "failed_link": 2  
}
```

Diagnosis:

Verify each link in the chain

```
agentauth chain verify \
```

```
  --poa-file poa.json \
```

```
  --verbose
```

Output:

Link 1: VALID (Board → CFO)

```
# Link 2:  INVALID (CFO → Manager)

#   - Signature verification failed

#   - Expected key: Ed25519:abc123...

#   - Actual key: Ed25519:xyz789...

# Link 3: Not reached
```

Solution: Link 2 was signed with wrong key. Recreate delegation from CFO with correct signing key.

17.1.2 Issue: “Transaction Exceeds Liability

Cap”

Symptoms:

```
{

  "error": "constraint_violation",

  "constraint_type": "liability_cap",
```



```

    "requested_amount": {"currency": "EUR", "amount": 75000},

    "authorized_amount": {"currency": "EUR", "amount": 50000}

}

```

Solutions:

1. **Request human approval** for this specific transaction
2. **Split transaction** into multiple sub-€50K transactions
(if appropriate)
3. **Request delegation upgrade** from principal

```
// Request human approval
```

```

func RequestApproval(tx *Transaction, poa *agentauth.PoA) error {

    approval := &ApprovalRequest{

        TransactionID: tx.ID,

        Amount: tx.Amount,

        AgentID: poa.Agent.Identity,

```

```

        PrincipalID: poa.Principal.Identity,

        Reason: fmt.Sprintf("Exceeds PoA liability cap of %v", poa.ConstraintCap),
    }

    // Send to approval workflow

    return approvalService.Submit(approval)
}

```

17.1.3 Issue: “Revocation Check Timeout”

Symptoms: System hangs for 30+ seconds during PoA verification.

Diagnosis:

```

# Check revocation service health

curl https://revocation.example.com/health

```

```
# Check network latency
```

```
ping revocation.example.com
```

```
# Check local cache
```

```
redis-cli GET "revocation:cache:poa-2025-001"
```

Solutions:

1. **Enable local caching** (TTL: 5 minutes):

```
verifier := agentauth.NewVerifier(&agentauth.VerifierConfig{  
  
    RevocationCache: &agentauth.CacheConfig{  
  
        Enabled: true,  
  
        TTL: 5 * time.Minute,  
  
        Backend: redisCache,  
  
    },
```

```
})
```

2. Configure degraded mode:

```
verifier := agentauth.NewVerifier(&agentauth.VerifierConfig{

    DegradedMode: &agentauth.DegradedModeConfig{

        Enabled: true,

        OnRevocationServiceDown: agentauth.AllowCached,

        MaxCacheAge: 15 * time.Minute,

    },

})
```

17.2 10.2 Frequently Asked Questions

17.2.1 Q: Can AgentAuth work without a central server?

A: Yes, with limitations.

AgentAuth supports **offline verification** (Degraded Mode): -

PoA tokens are self-contained - Signature verification works of-

fline - Constraint checking works offline - Revocation checks re-

quire cache or online service

For truly air-gapped systems: 1. Pre-populate revocation cache

2. Use short-lived PoAs (e.g., 1-hour validity) 3. Accept risk of

delayed revocation

17.2.2 Q: How does AgentAuth handle agent key rotation?

A: Built-in key rotation protocol.

```
// Rotate agent key
```

```
oldKeyID := "agent-key-2024"
```

```
newKeyID := "agent-key-2025"
```

```
// 1. Generate new key
```

```
newKey, _ := agentauth.GenerateEd25519Key()
```

```
// 2. Update entity profile
```

```
profile.PublicKeys = append(profile.PublicKeys, agentauth.PublicKey{
```

```
    KeyID: newKeyID,
```

```
    Algorithm: "Ed25519",

    KeyBytes: newKey.Public(),

    ValidFrom: time.Now().Add(7 * 24 * time.Hour), // 1 week overlap

})

// 3. Sign update with old key

signedUpdate, _ := profile.Sign(oldKey)

// 4. Publish update

entityService.Update(signedUpdate)

// 5. After overlap period, revoke old key

entityService.RevokeKey(oldKeyID)
```

17.2.3 Q: What happens if the principal's key is compromised?

A: Emergency revocation protocol.

1. **Immediate:** Report compromise to revocation authority
2. **Fast (5 min):** All PoAs issued by compromised key are
revoked
3. **Cascade:** All delegated PoAs also revoked
4. **Recovery (1 day):** New key issued, delegations recreated

Prevention: - Use HSM for principal keys - Enable multi-
signature for high-value principals - Regular key rotation

17.2.4 Q: Can PoA tokens be transferred?

A: No, by design.

PoA tokens are: - Bound to specific agent identity - Bound to specific principal - Non-transferable

Attempting transfer results in signature verification failure.

Exception: Delegation to sub-agents (requires principal consent).

Chapter 18

Chapter 11: Future Directions

18.1 11.1 Post-Quantum Cryptogra- phy

AgentAuth is preparing for the post-quantum era.

18.1.1 Current Status

Vulnerable algorithms: - Ed25519 (broken by Shor's algorithm) - ECDSA P-256/384 (broken by Shor's algorithm)

Quantum-resistant algorithms: - Dilithium-3 (lattice-based)
- SPHINCS+ (hash-based)

18.1.2 Roadmap

Phase 1 (2026): Hybrid signatures

```
{  
  
  "signature": {  
  
    "algorithm": "hybrid",  
  
    "classical": {  
  
      "algorithm": "Ed25519",  
  
      "value": "..."  
    }  
  }  
}
```

```

    },

    "post_quantum": {

        "algorithm": "Dilithium-3",

        "value": "...

    }

}

}

```

Both signatures must verify for token to be valid.

Phase 2 (2027): Pure post-quantum

```

{

    "signature": {

        "algorithm": "Dilithium-3",

        "value": "...

    }

```

```
}
```

18.1.3 Implementation Guide

```
// Enable hybrid signing
```

```
signer := agentauth.NewSigner(&agentauth.SignerConfig{
```

```
    Mode: agentauth.HybridMode,
```

```
    ClassicalKey: ed25519Key,
```

```
    PostQuantumKey: dilithiumKey,
```

```
})
```

```
poa, _ := signer.Sign(poaData)
```

```
// Verifier automatically handles hybrid
```

```
verifier := agentauth.NewVerifier(&agentauth.VerifierConfig{
```

```

    SupportedAlgorithms: []string{"Ed25519", "Dilithium-3", "Hybrid"},
})

```

```

result, _ := verifier.Verify(ctx, poa)

```

18.2 11.2 Zero-Knowledge Proofs

Use case: Prove constraints without revealing values.

Example: Prove “transaction < limit” without revealing exact amount.

```

// Generate ZK proof

```

```

proof := agentauth.GenerateZKProof(&agentauth.ZKProofRequest{
    Statement: "transaction_amount < liability_cap",

```



```
PrivateInputs: map[string]interface{}{

    "transaction_amount": 45000,

},

PublicInputs: map[string]interface{}{

    "liability_cap": 50000,

},

})
```

```
// Verify ZK proof
```

```
valid := agentauth.VerifyZKProof(proof)
```

Status: Research phase, targeting 2027 release.

18.3 11.3 Multi-Agent Coordination

Vision: Agents negotiating with other agents.

Scenario: - Agent A wants to purchase from Agent B - Both

agents verify each other's authority - Transaction executes only

if both PoAs are valid

```
// Multi-agent transaction
```

```
func NegotiateTransaction(agentA, agentB *agentauth.Agent) (*Transaction, error)
```

```
// Both agents present PoAs
```

```
    poaA := agentA.GetPoA()
```

```
    poaB := agentB.GetPoA()
```

```
// Cross-verify
```

```
    validA, _ := agentB.VerifyCounterparty(poaA)
```

```
validB, _ := agentA.VerifyCounterparty(poaB)

if !validA || !validB {

    return nil, fmt.Errorf("mutual verification failed")

}

// Execute transaction

tx := &Transaction{

    Buyer: agentA.GetPrincipal(),

    Seller: agentB.GetPrincipal(),

    Amount: negotiatedAmount,

    Signatures: []Signature{

        agentA.Sign(txHash),

        agentB.Sign(txHash),
```

```
    },  
  
    }  
  
    return tx, nil  
  
}
```

Status: Experimental, available in AgentAuth v2.0 (Q2 2026).

Chapter 19

Conclusion: The Path

Forward

We stand at an inflection point in computing history. For the first time, software can act autonomously on behalf of humans—not just execute commands, but make decisions, sign contracts,

move money.

This power demands accountability. OAuth gave us access control. AgentAuth gives us legal accountability.

The techniques in this book—delegation chains, cryptographic proofs, liability constraints—are not theoretical. They are production-ready, battle-tested, and compliant with German, US, and EU law.

The choice is ours:

We can continue deploying agents with access tokens, hoping nothing goes wrong.

Or we can deploy agents with legal mandates, knowing exactly who is responsible when things do go wrong.

The agent's signature is not just a cryptographic value.

It is a bridge between silicon and law, between algorithm and accountability.

When we sign, we accept responsibility. When agents sign, civilization scales.

END OF MANUSCRIPT

Mauricio A. Fernandez Fernandez December 31, 2025

